

LangGraph Failure & Retries

By

Anand Kulkarni

anand.kulkarni1@zensar.com

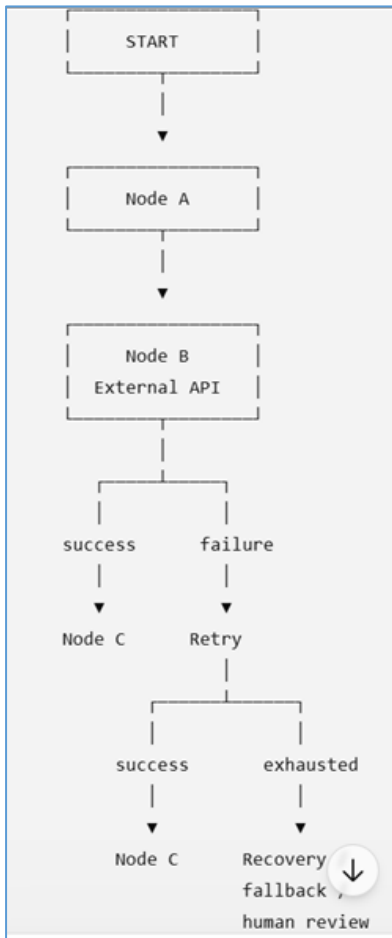
Table of Content

Module	Topic
Module 1:	Need of LangGraph Failure Recovery and Retries
Module 2:	LangGraph with retry support
Module 3:	Types of LangGraph Failures
Module 4:	Retry strategy
Module 5:	Apply retries using RetryPolicy
Module 6:	Apply Retry on appropriate exceptions
Module 7:	Realistic example of Retries
Module 8:	What if retries exhausted?

Need of LangGraph Failure Recovery and Retries

- LangGraph is designed for building stateful, multi-step, and often long-running LLM workflows.
- Since a graph may contain multiple nodes, external APIs, databases, tools, and LLM calls, failures are inevitable.
- Therefore, a robust LangGraph application therefore needs to answer questions such as:
 - 1) What happens when an LLM call fails?
 - 2) What happens when an API temporarily becomes unavailable?
 - 3) Should a failed node be retried?
 - 4) How many times should it be retried?
 - 5) How can the graph resume from a failure?

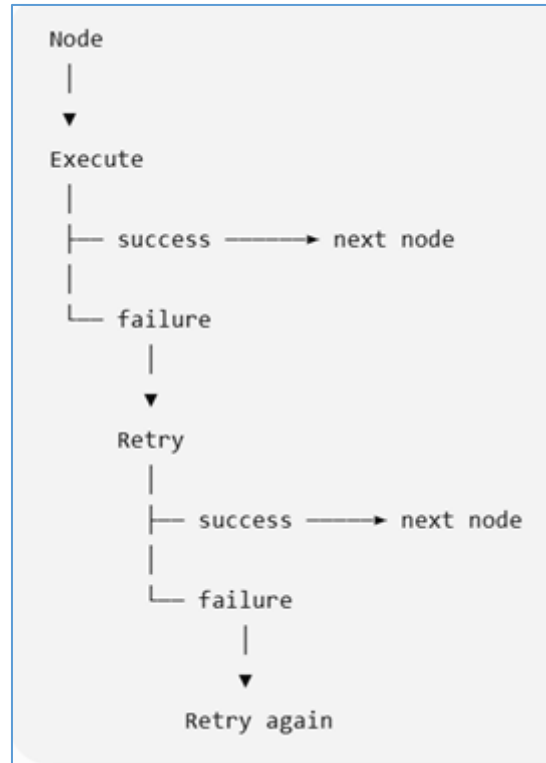
LangGraph with retry support



Types of LangGraph Failures

Failure Type	Examples
Transient failure	HTTP 503, Rate limit, Network timeout, Temporary database failure, LLM provider unavailable
Permanent failure	Invalid API key, Invalid input, Required database record doesn't exist, Malformed request
Tool failure	Database timeout, SMTP server timeout
Business-logic failure	Credit score = 450, Loan eligibility = rejected
Human-interruption failure	AI generated refund -> Amount > ₹50,000 -> Human approval required

Retry strategy



Apply retries using RetryPolicy

```
from langgraph.types import RetryPolicy
```

```
builder = StateGraph(State)
```

```
builder.add_node(
```

```
    "process",
```

```
    process_message,
```

```
    retry_policy=RetryPolicy(max_attempts=3)
```

```
)
```

```
builder.add_edge(START, "process")
```

```
builder.add_edge("process", END)
```

```
graph = builder.compile()
```

```
process()  
|  
├─ attempt 1 → failure  
|  
├─ attempt 2 → failure  
|  
└─ attempt 3 → success
```

Apply Retry on appropriate exceptions

```
retry_policy = RetryPolicy(  
    max_attempts=3,  
    retry_on=(  
        TimeoutError,  
        ConnectionError,  
    )  
)  
builder.add_node(  
    "external_api",  
    external_api,  
    retry_policy=retry_policy  
)
```

TimeoutError	→ retry
ConnectionError	→ retry
RateLimitError	→ retry
Invalid API key	→ don't retry
Invalid input	→ don't retry
Authentication failure	→ don't retry

Realistic example of Retries

```
import requests
```

```
class State(TypedDict):
```

```
    customer_id: str
```

```
    customer: dict
```

```
def get_customer(state):
```

```
    customer_id = state["customer_id"]
```

```
    response = requests.get(f"https://api.example.com/customers/{customer_id}", timeout=10)
```

```
    response.raise_for_status()
```

```
    return { "customer": response.json() }
```

Realistic example of Retries

```
builder = StateGraph(State)
```

```
builder.add_node("get_customer", get_customer,
```

```
    retry_policy=RetryPolicy(
```

```
        max_attempts=3
```

```
    )
```

```
)
```

```
builder.add_edge(START, "get_customer")
```

```
builder.add_edge("get_customer", END)
```

```
graph = builder.compile()
```

What if retries exhausted?

Option	Workflow
Option A – Fail the workflow	Failure -> stop -> alert
Option B – Fallback service	Primary API -> failed -> backup API
Option C – Human intervention	Failed -> Human intervention
Option D – Save state and resume later	Failed -> checkpointer -> resume later

Thank you!!